

Anàlisi i Classificació de Monstres de Dungeons & Dragons 5e

Pràctica Final — Resultats d'Aprenentatge (RA) — Intel·ligència Artificial

Javier Pedragosa Lozano

Curs 2025–2026 · INS Daniel Blanxart · CFGS Desenvolupament d'Aplicacions Web

1. Introducció

L'objectiu d'aquesta pràctica és aplicar els models de machine learning treballats a classe sobre un dataset real per extreure'n coneixement i construir una aplicació web funcional que permeti interactuar amb els models. He un dataset de monstres de *Dungeons & Dragons 5a Edició* per la seva riquesa en variables numèriques i categòriques.

El dataset conté 762 monstres amb atributs com el *Challenge Rating* (CR), Armadura (AC), Punts de Vida (HP), les sis estadístiques bàsiques (STR, DEX, CON, INT, WIS, CHA), el tipus de criatura, la mida i si és llegendari. S'han aplicat 5 models: Naive Bayes, Arbre de Decisions, K-Nearest Neighbors (KNN), Perceptró i Xarxa Neuronal (MLP).

2. Dataset

2.1. Descripció

El dataset `dnd_monsters.csv` s'ha obtingut de Kaggle (DnD 5e Monster Manual Stats). Conté 762 files i 17 columnes, de les quals les més rellevants per al modelatge són:

- **cr**: Challenge Rating (dificultat del monstre). Valors de 0 a 30, incloent fraccions (1/8, 1/4, 1/2).
- **ac**: Armadura (Armor Class).
- **hp**: Punts de vida.
- **str, dex, con, int, wis, cha**: Les 6 estadístiques bàsiques del joc.
- **type**: Tipus de criatura (dragon, humanoid, undead, beast...).
- **size**: Mida (Tiny, Small, Medium, Large, Huge, Gargantuan).
- **legendary**: Indica si el monstre té accions llegendàries.

2.2. Preprocessament

S'han realitzat els següents passos de neteja i transformació:

- Conversió del CR de fracció textual (1/4, 1/2) a valor float.
- Eliminació de files amb valors nuls en les sis estadístiques principals, resultant en 401 monstres amb dades completes.
- Eliminació de duplicats per nom.
- Simplificació del camp *type* en 14 categories (humanoid, undead, beast, dragon, fiend, construct, monstrosity, aberration, elemental, fey, giant, plant, ooze, celestial, other).
- Creació de la variable `cr_bracket` amb tres valors: Fàcil ($CR \leq 4$), Mitjà ($CR \leq 10$), Difícil ($CR > 10$).
- Creació de la variable binària `is_legendary`.
- Codificació de variables categòriques amb `LabelEncoder`.
- Escalat de features amb `StandardScaler` per als models sensibles a l'escala (KNN, Perceptró, Xarxa Neuronal).

3. Models aplicats

3.1. Naive Bayes — Classificació del tipus de monstre

S'ha aplicat el model **GaussianNB** per classificar el tipus simplificat de monstre (14 categories) a partir de les variables CR, AC, HP i les 6 estadístiques. S'ha triat Gaussian Naive Bayes per la naturalesa contínua de totes les features.

Variables	d'entrada:	cr,	ac,	hp,	str,	dex,	con,	int,	wis,	cha
Variable	objectiu:	type_simple				(14		categories)		
Divisió: 80% entrenament / 20% test										

El model ofereix un rendiment raonable, especialment en categories ben diferenciades com *dragon* (stats molt elevades) o *ooze*. La superposició entre humanoids i fiends redueix l'accuracy global.

3.2. Arbre de Decisions — Bracket de dificultat

S'ha entrenat un **DecisionTreeClassifier** amb profunditat màxima 6 per predir el bracket de dificultat (Fàcil/Mitjà/Difícil) a partir de les estadístiques AC, HP i les 6 bàsiques. La variable CR s'exclou intencionalment per evitar data leakage, ja que el bracket es deriva directament del CR.

Variables	d'entrada:	ac,	hp,	str,	dex,	con,	int,	wis,	cha
Variable	objectiu:	cr_bracket				(3 classes)			
Profunditat màxima: 6 min samples leaf: 3									

L'arbre identifica correctament que els monstres difícils tendeixen a tenir HP i STR molt elevats. El gràfic d'importància de features mostra que HP i CON són les variables més discriminatòries.

3.3. K-Nearest Neighbors (KNN) — Bracket de dificultat (CR)

S'ha aplicat **KNeighborsClassifier** per classificar el bracket de dificultat del monstre (Fàcil/Mitjà/Difícil) a partir de les estadístiques AC, HP i les 6 bàsiques, les mateixes features que l'arbre de decisions per permetre una comparació directa. Les features

s'han escalat amb StandardScaler, ja que KNN és sensible a l'escala de les dades.

Variables d'entrada: ac, hp, str, dex, con, int, wis, cha

Variable	objectiu:	cr_bracket	(3	classes)
-----------------	------------------	------------	----	----------

Selecció de k: validació creuada 5-fold sobre $k \in [1, 20]$

El millor valor de k trobat per validació creuada ha estat $k=4$. KNN identifica el bracket correctament buscant els 4 monstres més similars en l'espai de features normalitzat. En comparació amb l'arbre de decisions, KNN no genera regles explícites però s'adapta bé a fronteres de decisió irregulars.

3.4. Perceptró — Classificació binària (és llegendari?)

S'ha aplicat el **Perceptron** de scikit-learn per determinar si un monstre és llegendari o no, basant-se en totes les seves estadístiques. El dataset és molt desequilibrat (43 llegendaris de 401), i s'ha aplicat `stratify` al split per mantenir la proporció.

Variables d'entrada: cr, ac, hp, str, dex, con, int, wis, cha

Variable	objectiu:	is_legendary	(0	/	1)
-----------------	------------------	--------------	----	---	----

Hiperparàmetres: eta0=0.1, max_iter=5000, tol=1e-4

El Perceptró aconseguix una bona accuracy global però amb dificultat en la classe minoritària (llegendaris). Això és esperat: el Perceptró és un classificador lineal simple que no disposa de mecanismes per gestionar el desequilibri de classes.

3.5. Xarxa Neuronal (MLP) — Classificació de la mida

S'ha entrenat un **MLPClassifier** amb arquitectura de 3 capes ocultes (128–64–32 neurones) per classificar la mida del monstre (Tiny, Small, Medium, Large, Huge, Gargantuan). S'ha activat *early stopping* per evitar sobreajustament.

Variables d'entrada: cr, ac, hp, str, dex, con, int, wis, cha

Variable	objectiu:	size	(6	classes)
----------	-----------	------	----	----------

Arquitectura: (128, 64, 32) | activation=relu | learning_rate=0.001

La xarxa neuronal ofereix el millor rendiment en categories ben representades (Medium, Large). Les categories extremes (Tiny, Gargantuan) presenten més error degut a la seva baixa representació al dataset. La corba de pèrdua mostra una convergència ràpida i estable.

4. Comparació dels models

Naive Bayes	Classificació del tipus	Accuracy	~45–55%
Arbre de Decisions	Bracket CR (3 classes)	Accuracy	~75–85%
KNN	Bracket CR (3 classes)	Accuracy	~85–92%
Perceptró	Llegendari? (binari)	Accuracy	~88–92%
Xarxa Neuronal (MLP)	Classificació mida	Accuracy	~60–70%

L'arbre de decisions és el model amb millor rendiment en la seva tasca. Això té sentit perquè les regles de disseny de D&D segueixen patrons discrets i jeràrquics: monstres amb HP molt alts i bons stats gairebé sempre seran difícils, i l'arbre captura exactament aquest tipus de regles. El Perceptró aconsegueix una accuracy elevada però enganyosa per l'alt desequilibri de classes.

5. Aplicació web (Flask)

S'ha implementat una aplicació web amb **Flask** que ofereix tres seccions principals:

- **Dataset:** Visualització paginada del dataset complet amb cerca en temps real. Mostra un resum estadístic (total de monstres, tipus, CR màxim, legendaris).
- **Exploració:** Gràfics generats durant el preprocessament: distribució del CR, distribució de tipus, matriu de correlació entre stats i scatterplot HP vs. CR.
- **Models:** Interfície interactiva per fer prediccions amb qualsevol dels 5 models. L'usuari introdueix les estadístiques d'un monstre, selecciona el model i obté la

predicció amb probabilitats. També es mostren les matrius de confusió i gràfics de rendiment de cada model.

Els models s'han guardat en format `.pkl` amb `pickle` i es carreguen a l'inici de l'aplicació. Les prediccions es realitzen via una API REST (`/api/predict`) que retorna JSON. El frontend consumeix aquesta API amb `fetch`.

Per executar l'aplicació:

```
cd dnd_ml && python3 app.py  
Accedir a: http://localhost:5000
```

6. Conclusions

Aquesta pràctica ha permès aplicar i comparar els models de machine learning en un context real i temàticament interessant. Alguns aprenentatges destacables:

- El **preprocessament** és crític: sense netejar el dataset i gestionar els valors nuls, la majoria de models fallaven o donaven resultats sense sentit.
- L'elecció de la **variable objectiu** determina molt el rendiment del model. Classificar 14 tipus és molt més difícil que classificar 3 brackets de CR.
- El **desequilibri de classes** (pocs llegendaris) afecta molt els classificadors simples com el Perceptró.
- Els models més complexos (MLP) no sempre superen els simples (arbre de decisions) quan les dades segueixen patrons estructurats.
- La **integració amb Flask** permet fer el model accessible sense necessitat de coneixements de Python per part de l'usuari.